Reforge: Specification-Driven Regenerative Coding

Hrvoje Abraham • ahrvoje@gmail.com • Zagreb, Croatia • April 4, 2026 • Version 1

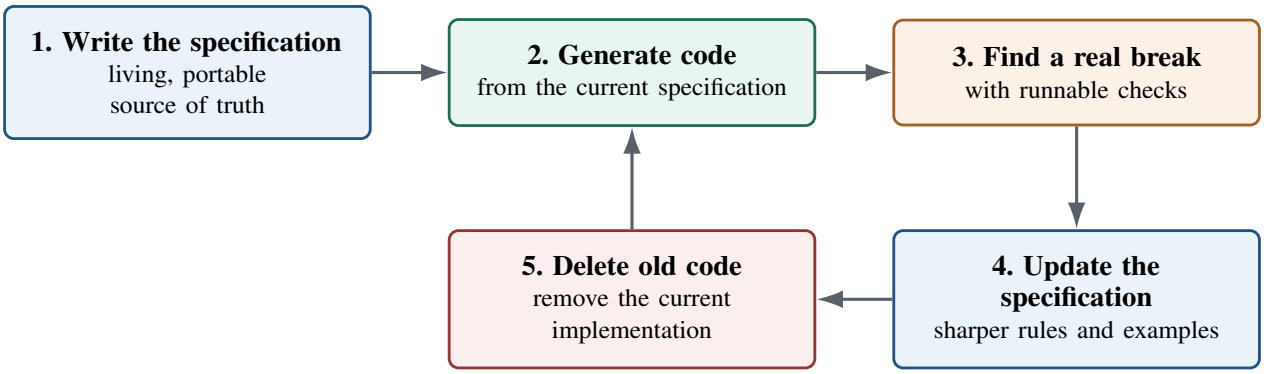

Fig. 1. Reforge at a glance: specify, generate, break, update, and regenerate.

Abstract

AI code generation is fast, but reliability still collapses when important requirements remain implicit. We present Reforge, a specification-driven form of regenerative coding. By regenerative coding we mean building and rebuilding code from non-code resources, with the living specification at the heart of the process rather than hidden memory in the previous implementation. Reforge is not merely a static `spec.md` plus one generation pass and one review pass; it assigns coding, review, benchmark, and specification-update roles around a living specification that acts as the sole durable memory of the loop and as a first-class long-term product. The coding agent generates from the current specification, the review agent tries to falsify success with runnable behavioral, performance, or structural checks, validated findings are rewritten as portable specification clauses, the old implementation is deleted, and the next round is regenerated from the revised specification alone. A worked binary64 `log1p(x)` case study traces explicit specification versions, named counterexamples, and one important non-counterexample discovery that sharpened the validation regime. Reforge aims at high-confidence engineering and often yields shorter, more coherent implementations, but it remains a disciplined reliability method rather than a proof system.

I. Introduction

AI coding systems are already good enough to compress weeks of routine implementation into hours, but production work still fails at the trust boundary. A strong model can write a plausible repository quickly, yet teams often discover later that edge cases were never stated, performance expectations were only implied, and the project layout itself has become too tangled for either humans or models to reason about safely. The result is speed without durable confidence.

That gap is easy to underestimate because small single-purpose scripts can tolerate far more implicitness than durable software can. A short one-time-task script may still be serious code, but it is not usually expected to survive broad compatibility demands, unclear future requirements, or long-term maintenance across teams and platforms. Reusable software is different. Once the target is meant to persist, the missing requirements have to be written down somewhere durable.

Core idea. Write a living specification, generate code from it, try to break the code with runnable counterexamples, rewrite what was learned as absolute portable requirements, delete the old implementation, and regenerate from the improved specification alone.

We call this workflow **Reforge**. Reforge is **specification-driven**: the living specification is the explicit driver of each coding and recoding round, not a loose background hint. More broadly, Reforge is **regenerative coding**: code is generated from non-code resources, chiefly the specification, instead of being carried forward mainly through patch memory in the previous implementation. The baseline it improves on is mundane: keep one `spec.md`, ask one LLM to generate code from it, and ask another LLM to review the result. That already helps, but it is not yet a full workflow. It does not tell the team who may change the specification, what counts as an accepted bug, how stakeholder concerns become executable checks, when old code must be deleted, or what honest stopping looks like. Reforge begins where that baseline stops: with explicit agentic roles, artifact handoffs, evidence rules, specification updates, deletion, regeneration, and review budgets.

Reforge is deliberately adversarial. The coding agent tries to satisfy the current specification, while the review agent tries to falsify that claim with concrete evidence. Those roles are opposed, but only in service of the same target. The reviewer is not asked for taste, style, or general skepticism. It is asked to produce confirmed failures backed by runnable tests, benchmarks, or structural checks with explicit pass/fail assertions.

That asymmetry is intentional. The generator is rewarded for satisfying the specification. The reviewer is rewarded for producing falsifying evidence. Both roles are constrained by the same contract: a concern only counts when it becomes either an explicit executable check or a precise, implementation-independent clarification.

That distinction matters because ordinary generate-review-patch cycles often hide learning in the current codebase instead

of in a durable artifact. A patch can silence one complaint while quietly weakening another property. A long chat transcript can accumulate clever observations, but if those observations never become implementation-independent requirements, the next reimplementation starts from amnesia. Reforge tries to prevent that drift by forcing validated learning into `spec.md`, then deleting the code that previously carried the lesson implicitly.

The specification therefore matters twice over. It is the memory of the loop, and it is also the collaboration layer where design intent, project priorities, support failures, architectural constraints, platform limits, legal concerns, localization details, and AI-discovered edge cases can be reconciled into one portable source of truth. Reforge treats the production of that strengthened specification as part of the product, not merely as overhead on the way to code.

Reforge is not presented as a new origin story for CEGIS itself. The lineage runs from counterexample-driven learning and abstraction refinement through the Sketch-era synthesis loop and the later naming of CEGIS [1], [2], [3], [4], [5], [6]. The broader grouped comparison with repair loops, validator-backed refinement, multi-agent generation, and specification-to-code systems is kept in the related-work section. The contribution here is narrower and more operational: **Reforge is a specification-driven, counterexample-guided form of regenerative coding for full implementations or modules in which the living specification is the central durable memory and a first-class long-term product.**

The rest of the paper explains why naive review loops fail, defines Reforge in operational terms as specification-driven regenerative coding, shows the prompt discipline that commonly drives the coding, review, and specification-update roles, explains how correctness, performance, and project structure enter the same evidence contract, and then walks through a worked binary64 `log1p(x)` example with explicit specification versions and counterexamples.

II. Related Work and Positioning

The historical lineage of Reforge is clear. Angluin's exact-learning model already used conjecture, counterexample, and refinement as an abstract pattern [1]. Clarke *et al.* made counterexample-guided refinement central in CEGAR [2]. Solar-Lezama *et al.*'s Sketch paper gives an early clear program-synthesis instance of the synthesize/verify/counterexample loop [3], and Solar-Lezama's dissertation explicitly names the method CEGIS [4]. Jha *et al.* and Gulwani *et al.* then place CEGIS inside a broader oracle-guided synthesis lineage [5], [6]. Reforge builds on that family of ideas, but applies it to present-day AI code generation rather than claiming to redefine CEGIS.

The closest modern adjacent work is repair- and refinement-oriented. Orvalho *et al.* combine MaxSAT fault localization, LLM repair, and counterexample feedback for buggy student programs, and their public repository reflects the same repair-centric structure [7], [8]. Self-Debugging and Reflexion show that iterative execution feedback and textual memory can improve later attempts [9], [10]. SCAFFOLD-CEGIS, LLMLOOP, and BACE all attack the broader problem that iterative loops can degrade important properties unless they are constrained by stronger checks or better feedback [11], [12], [13]. Reforge overlaps with all of these lines, but it moves the long-term memory of the process into the specification rather than into evolving code, tests, or agent-local reflections.

Another nearby cluster studies full generation from specifications, specialized roles, or richer orchestrations. AgentCoder and AlphaCodium use multiple agents or staged flows to harden generated code [14], [15]. Requirements are All You Need pushes structured requirements through a prompt pipeline into code artifacts [16]. Spec2RTL-Agent, MACOG, and HDLFORGE show how multi-agent systems can generate larger artifacts from complex specifications while leaning on validation and specialization [17], [18], [19]. SR-Eval demonstrates that iterative requirement change remains a hard setting for current models, and Kobaladze *et al.* place these approaches inside the broader synthesis landscape [20], [21]. Reforge is compatible with these systems, but its differentiator is the insistence that validated learning migrate into a portable specification that can drive later regeneration after the current implementation has been deleted. In that sense, Reforge is better understood as specification-driven regenerative coding than as ordinary iterative patching.

III. Why Naive AI Code Review Loops Fail

The most common failure pattern in AI coding is not that models never improve. It is that teams accept improvement without knowing where the new knowledge lives. One-shot prompting leaves too much implicit. Even the slightly stronger baseline of "one specification file, one generation prompt, one review prompt" still leaves unanswered where validated learning should persist. Ordinary patch loops often convert vague criticism into local edits without deciding whether the criticism was actually true, whether it should become a durable rule, or whether the fix weakened something elsewhere.

Review models make this worse when they are rewarded for sounding insightful instead of for proving failure. A long list of suspicions can feel productive, but in Reforge terms most of those comments are not yet knowledge. Until a concern becomes an executable check or a precise forward-looking clarification, it cannot reliably survive deletion of the current codebase.

This is why Reforge treats deletion as a feature rather than a cost. If a requirement was real, it should survive as specification text. If it only survived because the old code happened to contain a brittle workaround, the next regeneration exposes that omission immediately.

IV. A Practical Counterexample-Guided Loop

Reforge is meant to be executable as a reusable human-and-agent routine, not only admired as a slogan. The workflow begins with a human-written `spec.md` that states the task, constraints, success criteria, performance expectations, and structural expectations in implementation-agnostic terms wherever possible. A coding agent then generates from that

TABLE I
POSITIONING REFORGE RELATIVE TO NEARBY WORKFLOW FAMILIES.

Line of work	Main durable memory	Typical evidence channel	Relation to Reforge
Repair-centric CEGIS and repair loops [7], [8]	Evolving program plus localized fix context	Counterexamples against the repaired artifact	Very close in spirit, but centered on repairing an existing program rather than deleting it and regenerating from a living specification.
Iterative self-correction and reflection [9], [10], [12], [13], [11]	Intermediate code, tests, or agent memory	Execution feedback, static analysis, mutation, or reflective text	Shares the value of feedback loops, but does not usually make the specification the sole durable memory.
Multi-agent full generation [14], [15], [17], [18], [19]	Plans, agents, blackboards, evolving code, or intermediate representations	Specialized validators, staged generation, or agent coordination	Shows that reliable full generation is feasible, but the enduring artifact is usually the evolving system rather than the specification alone.
Requirements-to-code and evolving-requirement systems [16], [20], [21]	Requirements, prompts, and generated artifacts	Structured prompting, evaluation under requirement change, or broad synthesis framing	Establishes that specifications matter, but not the delete-and-regenerate discipline that makes knowledge migration testable.
Reforge	**Portable living specification**	**Runnable counterexamples, benchmarks, and structural checks**	**Validated findings become future-facing specification clauses; the implementation is disposable.**

TABLE II
THREE COMMON AI CODING PATTERNS. REFORGE DIFFERS BY MAKING VALIDATED LEARNING SURVIVE CODE DELETION.

Pattern	Where memory accumulates	Typical failure mode	Practical consequence
One-shot prompting	In the model's unstated guess about the task	Hidden requirements never become explicit; confidence is based on plausibility	Fast start, weak transfer, brittle reimplementation
Generate-review-patch	In the current code, local tests, and chat history	Comments can be vague, contradictory, or overfit to the current code shape	Many edits, little durable knowledge, hard-to-audit progress
Reforge	**In the living specification and accepted counterexamples**	**Work stalls only when evidence cannot be produced or the specification is infeasible**	**Learning transfers to future regeneration across models, stacks, and platforms**

document. That is the specification-driven part: the specification explicitly drives the round. The loop is regenerative because accepted learning is rewritten into non-code artifacts before the next implementation is built. A review agent, operating in an intentionally adversarial role, tries to break the implementation under explicit criteria. The coding agent updates the specification first with durable lessons discovered during implementation and testing; the review side then adds its own validated clarifications and counterexamples. Only after that update step does the team delete the old code and regenerate.

At the time of writing, a practical session default was Opus 4.6 Max as coding agent and GPT-5.4 extra-high as reviewer and counterexample finder. That note is time-stamped, not doctrinal: Reforge is designed to remain model-agnostic as stronger generators, reviewers, and domain tools arrive.

The operational loop can be stated plainly:

1) Consolidate stakeholder intent into a current specification.
2) Generate an implementation from that specification.
3) Attempt to falsify the implementation with runnable behavioral, performance, or structural checks.
4) Rewrite validated findings as absolute, forward-looking specification clauses.
5) Delete the old implementation.

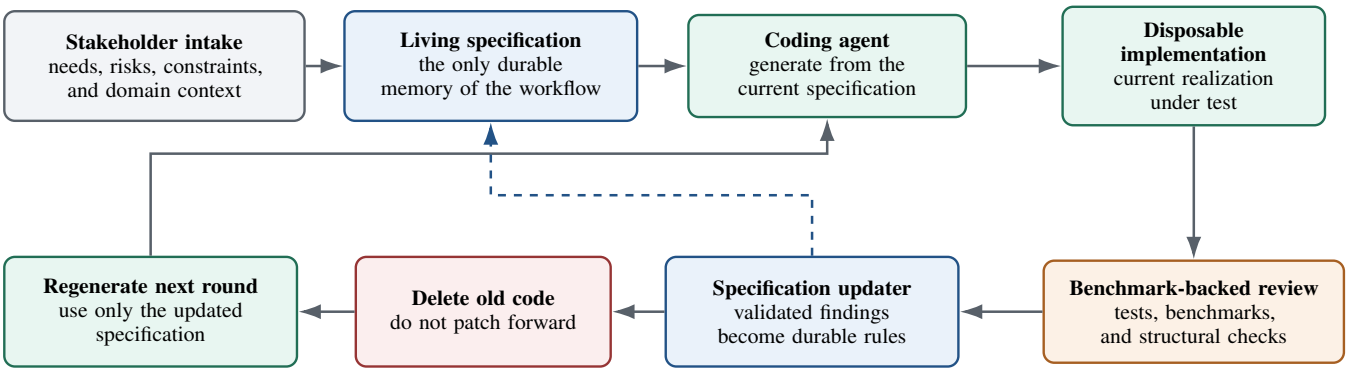

Fig. 2. Operational view of Reforge. Stakeholder concerns are translated into specification clauses and executable checks; the specification persists while implementations are deleted and regenerated.

6) Regenerate from the specification alone.
7) Stop when no confirmed bug is found within the review budget.

These operational prompts are part of the workflow, but they are not the workflow by themselves. What makes Reforge agentic is the larger operating contract around those prompts: which role may claim what, how raw concerns become executable checks, how the specification is mutated, and when regeneration is allowed to begin.

These prompts are intentionally terse. A long prompt can help a little, but Reforge does not depend on prompt verbosity to enforce discipline. The workflow itself carries most of that load: stakeholder concerns are translated into hypotheses, the hypotheses become executable checks or precise clarifications, the coding agent updates the specification first, the review side adds its own validated findings second, the specification-updater checks the resulting draft for ambiguity, inconsistency, and infeasible accumulation, and regeneration is allowed only after the revised specification is accepted.

Several actor roles matter even when the same person or model fills more than one of them. Stakeholders such as developers, project managers, designers, support, legal reviewers, platform owners, sales, or end users may originate concerns. Those concerns enter the loop as a pipeline: concern $\rightarrow$ hypothesis or missing requirement $\rightarrow$ executable check or precise clarification $\rightarrow$ accepted specification update if validated. The coding and review agents operationalize those concerns into checks. A benchmark or test agent can own oracle creation, search-region generation, and performance measurement. A specification-updater role decides what portable text should survive after each round and should explicitly reject or escalate ambiguous, inconsistent, or infeasible requirement bundles before more code is generated. A human approver ultimately decides whether the current review budget is strong enough to stop.

The adversarial setup is useful because it converts model capability into pressure. The generator is rewarded for satisfying the spec. The reviewer is rewarded for producing falsifying evidence. Both roles are disciplined by the same contract: concerns

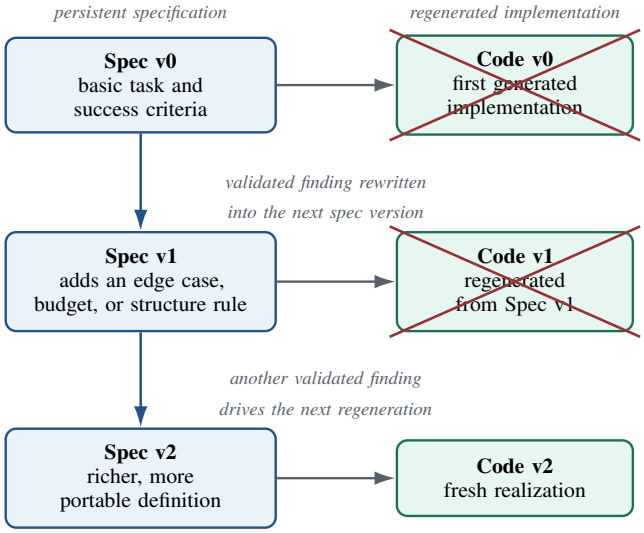

Fig. 3. Validated findings migrate into the next specification version; intermediate implementations do not become the long-term memory of the process.

only count when they can be translated into explicit checks or into precise, implementation-independent clarifications. That keeps review from degenerating into vague negativity while still pushing the generator hard. It also makes the incentives legible enough that a human-and-agent team can run the loop deliberately rather than by vague intuition, even when the same model family fills more than one operational role.

V. Counterexamples as the Unit of Progress

Confirmed counterexample. In Reforge, a counterexample is a specific input, state, execution path, benchmark result, or structural scenario that violates a concrete claim in the specification and is backed by a runnable failing assertion. If the reviewer cannot point to that explicit failure, it has not yet found a bug for the purposes of the loop.

This evidence rule is deliberately strict. “This might be wrong” is not enough. “The code looks slow” is not enough. “The modules feel messy” is not enough. Those may all be good

TABLE III
REPRESENTATIVE OPERATIONAL PROMPTS. THE WORKFLOW, NOT PROMPT LENGTH, SUPPLIES MOST OF THE DISCIPLINE.

Role	Representative prompt
Coding agent	`Generate the implementation of the function as specified in spec.md.`
Review agent	`Make deep review of the implementation targeting the specification defined in spec.md and provide explicit reproducable counterexamples. Do not make speculative, academic or stylistic remarks, but report only bugs reproducable by running through the implementation. Update spec.md counterexamples set accordingly, and report which specification ambiguity or inconsistency could have contributed to the bug. If explicit counterexample which contradicts the specification is not found, there is no proof of the bug.`
Specification updater	`Improve the specification file spec.md to fulfill all of the following requirements: 1. ... 2. ... 3. ... Then check for the specification consistency, ambiguity, and grill me for further inputs if needed or the requirements accumulate into an infeasible task.`

TABLE IV
ACTOR AND ARTIFACT MAP FOR AN OPERATIONAL REFORGE SESSION.

Role	Inputs it may raise	What counts as acceptable output	Artifact passed forward
Stakeholder intake	Product intent, UX constraints, support failures, domain rules, legal or platform limits	Concrete requirements, risks, or ambiguities worth resolving	Candidate specification clauses or review hypotheses
Coding agent	Current specification and prior accepted findings	Implementation attempt plus implementation-side clarifications that remain meaningful after code deletion	Disposable code and portable specification edits
Review agent	Current specification, implementation under test, search budget	Confirmed failures only: runnable tests, benchmarks, or structural checks with explicit assertions	Counterexamples and validated clarification requests
Benchmark or test agent	Review hypotheses and oracle definitions	Reproducible harnesses, search regions, performance measurements, and structural inspections	Evidence artifacts that justify accepting or rejecting a claim
Specification updater	Accepted evidence from coding and review sides	Absolute requirements, prohibitions, examples, budgets, or decomposition rules	Next `spec.md` version
Human approver	Current specification, evidence log, and risk tolerance	Stop/continue decision based on the stated review budget	Release decision or another regeneration round

suspicions, but they only become accepted knowledge after being translated into something executable or into a precise portable clarification. This is the same contract that governs both sides of the adversarial setup: the generator does not win by sounding plausible, and the reviewer does not win by sounding sharp. For failed input/output claims, runnable counterexamples are mandatory. For ambiguity or structural lessons discovered during implementation, the standard is slightly different: there may be no single decisive failing input, but the resulting update still has to be written as a standalone future requirement rather than as commentary on the just-deleted code. This is also why the operational review prompt is so blunt about prohibiting speculative, academic, and stylistic remarks. Reforge review is trying to break the code under explicit criteria, not to perform a general literature seminar or taste audit.

The reward for that discipline is that one failure can harden every future regeneration. Once a reviewer produces a real break, the team does not merely patch that instance. It upgrades the specification so later generators inherit the lesson without relying on hidden memory in the old code, the old chat, or the old test harness.

VI. PERFORMANCE AND STRUCTURE AS FIRST-CLASS CONSTRAINTS

Correct output is only part of whether AI-generated code is usable. A program can satisfy visible examples while missing performance budgets, exhausting memory, or collapsing into

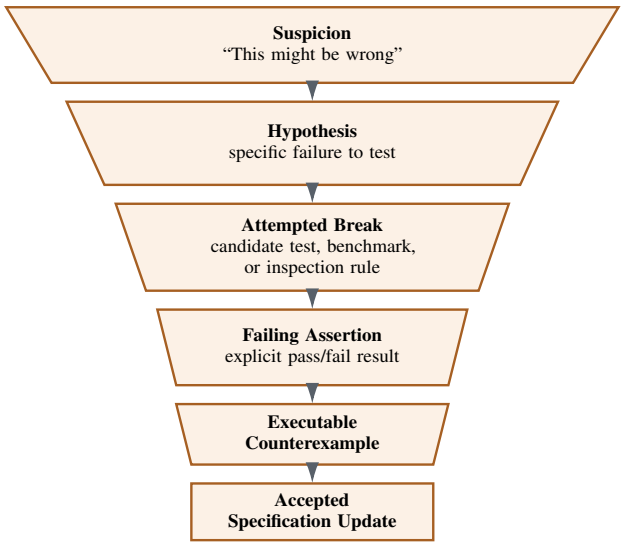

Fig. 4. Review comments become durable knowledge only after they narrow into executable counterexamples or precise specification updates.

TABLE V
THREE KINDS OF ACCEPTED REFORGE EVIDENCE.

Type	Required evidence	Example durable rewrite
Correctness	Failing assertion on a named input, state, or trace	"For finite `x > -1`, the result must remain finite and stay within 2 ULP of the oracle on the declared validation regions."
Performance	Benchmark with explicit budget and measured violation	"The median latency on the stated workload must stay below the declared budget on the stated hardware class."
Structure	Reproducible structural check against explicit decomposition rules	"No generated module may exceed the declared semantic scope or file-size budget without an explicit reason in the specification."

a project structure that no longer fits within realistic model context and output limits. Reforge therefore treats performance and structure as legitimate targets of the same loop.

Performance constraints belong in the specification when they are truly part of the problem. They should be written as explicit budgets on representative workloads, hardware classes, or resource ceilings, not as vague aspirations such as "be fast." A performance complaint then enters the loop only after a benchmark crosses a stated boundary.

Structural constraints matter for a different reason. LLMs do not merely fail because logic is wrong; they also fail when codebases become too diffuse, too large, or too semantically mixed for the model to keep the whole design in working memory. Reforge therefore allows the specification to constrain module boundaries, file-size ceilings, dependency rules, and semantic scope. These are not style rules. They are tractability rules for future generation, review, and maintenance.

The same caution applies in reverse: structure should not become architecture theater. If a simple repository organization is adequate, Reforge should preserve that simplicity. The point is to make future regeneration coherent, not to force gratuitous abstraction into the specification.

VII. WHY THE SPECIFICATION, NOT THE CODE, MUST PERSIST

Deleting the previous implementation is the move that keeps Reforge honest. Without deletion, teams can always hope that the important lesson "must be somewhere in the code." With deletion, that hope is tested immediately. If the next generation cannot reproduce the property from the updated specification, then the lesson was never truly captured.

This makes the specification more than a prompt. It becomes the durable interface for future work: the document that stronger future models, different target languages, different platforms, or different human teams can inherit without needing the local debugging history of today's repository. Reforge therefore treats specification writing as part of the primary product, not merely as preparatory paperwork.

That perspective also turns the specification into a collaboration layer. Product intent, design considerations, support failures, legal constraints, localization details, platform limits, and AI-discovered edge cases can all be reconciled in one portable artifact. The code is the current realization. The specification is the enduring contract.

VIII. WORKED CASE STUDY: BINARY64 `LOG1P(X)`

The worked example is a binary64 implementation of `log1p(x)`. It is a good stress test because the interface is tiny while the numerical behavior near zero and near the singular boundary at $x = -1$ is unforgiving. The dominant constraint here is correctness; performance and structure matter only secondarily. That makes the specification changes easy to inspect, and it makes the example the paper's main concrete demonstration rather than a lightweight anecdote.

For the illustrative run, assume the time-stamped default pairing discussed earlier: Opus 4.6 Max as coding agent, GPT-5.4 extra-high as reviewer and counterexample finder, and the three-prompt operational set in Table III. The specification-updater step is human-guided in this example: it uses the short update prompt as a consistency and feasibility checklist while merging coding-side clarifications with validated reviewer evidence. The update order also follows the operational rule used throughout Reforge: the coding side records implementation-side clarifications first, the review side adds validated counterexamples and ambiguity reports second, and only then is the previous implementation discarded.

a) Version 0.: The initial specification was intentionally short:

- Input domain: binary64 `x`.

TABLE VI
WORKED-EXAMPLE ITERATION TRAIL FOR BINARY64 `LOG1P(X)`.

Version	What was learned	Evidence	Durable specification update
V0	Natural-language task statement was too weak near numerically delicate regions.	Early candidate collapses `log(1+x)` to `0.0` at `x = 1e-16`.	Add explicit oracle and a 2-ULP accuracy target instead of vague agreement.
V1	Correctness needs declared validation regions, not a single random pool.	Testing process repeatedly missed the hard near--1 region until the search space was partitioned.	Name the regions explicitly and require coverage over each of them.
V2	Boundary behavior must be executable, not implied.	Later candidate returns $-\infty$ for $x = -0.9999999999999999$, violating the "finite for finite `x > -1`" rule.	Add an explicit finiteness clause for every finite input above the singular boundary and add monotonicity checks on ordered grids.
V3	Final candidate survives the stated review budget.	60,000 sampled finite inputs across three regions, plus special-value checks and three 2,048-point monotonicity grids, found no confirmed bug; worst observed finite-case error was 1 ULP.	Stop under the stated budget, while keeping the conclusion short of proof.

- For finite `x > -1`, return a binary64 approximation of $\ln(1+x)$.
- For `x = -1`, return $-\infty$.
- For `x < -1` or `NaN`, return `NaN`.
- For `x = 0`, preserve signed zero.

b) Counterexample 1.: An early candidate used the tempting but unstable pattern "compute `log(1+x)` directly." For the exact binary64 input `x = 1e-16`, the intermediate sum `1+x` rounds to `1.0`, so the candidate returns `0.0`. A 100-digit Decimal oracle built from the exact binary64 input gives $y_{\text{ref}} \approx 9.9999999999999993 \times 10^{-17}$, so the specification-level assertion

$$\texttt{ulp_error}(y, y_{\text{ref}}) \leq 2$$

fails immediately. The problem was not subtle. The initial specification never told the generator what numerical standard to meet near zero.

c) Version 1 update.: The next specification version added three portable clauses: (1) define an explicit high-precision oracle for validation, (2) require a maximum error of 2 ULP on declared validation regions, and (3) keep the special-value and signed-zero rules explicit rather than implied.

d) Intermediate discovery.: During testing, another lesson appeared that was not itself a single decisive counterexample. An undifferentiated random test pool oversampled easy interior points and undersampled the dangerous neighborhoods near zero and near -1. The workflow therefore needed declared validation regions, not just "many random tests." This is exactly the kind of discovery that should become specification text even though it is not one failing input.

e) Version 2 update.: The specification was sharpened again: it now required coverage over three named regions (near zero, near -1, and moderate positive inputs), required monotone behavior on ordered validation grids over the supported finite domain slices, and kept the requirements independent of any particular polynomial, branch structure, or file layout.

f) Counterexample 2.: A later candidate improved the near-zero behavior but used a brittle boundary guard that treated values within 2^{-52} of -1 as the singular case. For the exact binary64 input $x = -0.9999999999999999$, that candidate returned $-\infty$ even though the input is finite and still satisfies `x > -1`. The exact binary64 oracle is finite, $y_{\text{ref}} \approx -36.73680056967710$, so the explicit assertion "finite `x > -1` must produce a finite result" fails. This counterexample sharpened the domain boundary rule from a natural-language intention into a hard executable condition.

g) Version 3.: The final worked-example specification was still short, but far stronger than Version 0:

- Preserve the special-value behavior above, including signed zero.
- Validate finite outputs against the declared high-precision oracle.
- Require error ≤ 2 ULP on declared near-zero, near--1, and moderate-positive validation regions.
- Require finite output for every finite `x > -1`.

- Require monotone behavior on the declared ordered validation grids.

h) Final review status.: For the final candidate, review used the same explicit specification clauses rather than hidden implementation knowledge. Finite cases were checked against a 100-digit Decimal oracle built from the exact binary64 input and rounded back to binary64 for ULP comparison. The review budget covered 60,000 sampled finite inputs split across near-zero, near--1, and moderate-positive regions, plus explicit checks for `NaN`, $-\infty$, the singular point at `x = -1`, signed zero, and three ordered monotonicity grids with 2,048 points each. Under that budget, no confirmed bug was found, and the worst observed finite-case error was 1 ULP.

i) Takeaway.: The value of the example is not that `log1p(x)` is special. It is that the loop made improvement legible. The first counterexample forced an oracle and a ULP budget into the specification. The intermediate discovery forced the validation regions into the specification. The second counterexample forced an executable boundary rule into the specification. By the end, the document that survived was not a pile of patch notes. It was a compact, portable definition that could guide the next implementation even if the current one were deleted entirely.

IX. Discussion and Limitations

Reforge depends on specification quality. If the target is incoherent, contradictory, or unrealistically ambitious, regeneration will not rescue it. The workflow is designed to extract disciplined performance from strong models, not to make impossible specifications feasible.

Reforge also depends on review depth. Some domains provide cheap oracles and crisp assertions; others do not. Numerical kernels, protocol logic, and many transformation tasks are unusually friendly to runnable counterexamples. Open-ended user experience goals or weakly measurable architectural qualities are harder. In those cases, the workflow still helps, but the evidence channel may rely more on structured inspections and less on decisive failing inputs.

There is also real cost. Reforge spends tokens, compute, and human attention on repeated regeneration and stronger review. That cost is worthwhile only when the task is important enough that durable confidence matters more than first-answer speed.

Finally, Reforge is not formal verification. "No confirmed bug found within the review budget" is a meaningful engineering statement, but it is not a proof of correctness. The workflow occupies the space between casual prompting and theorem-backed assurance: much stricter than informal patching, much lighter than full proof.

X. Conclusion

Reforge treats AI code generation as specification-driven regenerative coding under adversarial counterexample pressure. It is more than a static specification plus a generation prompt plus a review prompt: the coding agent writes against the current specification, the reviewer tries to falsify success with executable evidence, the specification-updater turns accepted learning into a coherent next draft, and the old implementation is deleted so the next round must stand on the specification alone.

That shift changes what the durable product is. The code remains important, but the long-term asset is the portable specification that captures correctness rules, performance budgets, structural limits, and named counterexamples in a form that future teams and future models can reuse. Reforge therefore offers a practical route to higher-confidence regenerative coding without claiming to turn software generation into proof.

References

[1] D. Angluin, "Learning regular sets from queries and counterexamples," *Information and Computation*, vol. 75, no. 2, pp. 87–106, 1987.

[2] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Computer Aided Verification*, 2000.

[3] A. Solar-Lezama, L. Tancau, R. Bodik, S. A. Seshia, and V. A. Saraswat, "Combinatorial sketching for finite programs," in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, 2006.

[4] A. Solar-Lezama, "Program synthesis by sketching," Ph.D. dissertation, Massachusetts Institute of Technology, 2008.

[5] S. Jha, S. Gulwani, S. A. Seshia, and A. Tiwari, "Oracle-guided component-based program synthesis," in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, 2010.

[6] S. Gulwani, O. Polozov, and R. Singh, "Program synthesis," *Foundations and Trends in Programming Languages*, vol. 4, no. 1–2, pp. 1–119, 2017.

[7] P. Orvalho, M. Janota, and V. Manquinho, "Counterexample guided program repair using zero-shot learning and MaxSAT-based fault localization," 2025, aAAI 2025 / arXiv:2502.07786.

[8] pmorvalho, "LLM-CEGIS-Repair repository," 2025, gitHub repository.

[9] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," 2023, arXiv:2304.05128.

[10] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," 2023, arXiv:2303.11366.

[11] Y. Chen, Y. Bian, H. Wang, S. Li, and Z. Cui, "SCAFFOLD-CEGIS: Preventing latent security degradation in LLM-driven iterative code refinement," 2026, arXiv:2603.08520.

[12] R. Ravi, D. Bradshaw, S. Ruberto, G. Jahangirova, and V. Terragni, "LLMLOOP: Improving LLM-generated code and tests through automated iterative feedback loops," 2026, iCSME 2025 accepted manuscript / arXiv:2603.23613.

[13] K. Silva and S. Perera, "BACE: LLM-based code generation through bayesian anchored co-evolution of code and test populations," 2026, gECCO 2026 / arXiv:2603.28653.

[14] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," 2023, arXiv:2312.13010.

[15] T. Ridnik, D. Kredo, and I. Friedman, "Code generation with AlphaCodium: From prompt engineering to flow engineering," 2024, arXiv:2401.08500.

[16] B. Wei, "Requirements are all you need: From requirements to code with LLMs," 2024, arXiv:2406.10101.

[17] Z. Yu *et al.*, "Spec2RTL-Agent: Automated hardware code generation from complex specifications using LLM agent systems," 2025, arXiv:2506.13905.

[18] R. N. H. Khan *et al.*, "Multi-agent code-orchestrated generation for reliable infrastructure-as-code," 2025, arXiv:2510.03902.

[19] A. Abdollahi *et al.*, "HDLFORGE: A two-stage multi-agent framework for efficient verilog code generation with adaptive model escalation," 2026, arXiv:2603.04646.

[20] Z. Zhan, S. Gao, R. Hu, and C. Gao, "SR-Eval: Evaluating LLMs on code generation under stepwise requirement refinement," 2025, arXiv:2509.18808.

[21] Z. Kobaladze, A. Arnania, and T. Sanikidze, "From provable correctness to probabilistic generation: A comparative review of program synthesis paradigms," 2025, arXiv:2508.00013.